Edward Curve Scalar Multiplication Over Finite Fields

Dustin Ray

1.a. Introduction

Computerized technology has revolutionized the world and the lives of almost everyone living in it. The digital age has brought forth many important advancements that have greatly improved the human condition on an unprecedented scale. The security behind this technology, however, requires a great deal of consideration and attention. Computers can cause harm on a scale equivalent to their benefit if they are not constructed and implemented carefully, and thus the security and robustness of these devices and the software that powers them is of tantamount importance.

Cryptography sits in front of the output of almost every computerized device today. Anything read from a screen or listened to from headphones or speakers has likely passed through some form of cryptography at one point. Optimization of modern software and algorithms in terms of performance and efficiency is a topic of great importance, but this is intrinsically dependent on the performance and efficiency of the cryptographic constructs that secure such software. Often times the most significant bottleneck to the runtime of any piece of software lies in the implementation of its cryptographic protocols, and the topic of research into finding faster and more efficient cryptographic methods is an area of intense research.

Improvements in efficiency and running times of cryptographic software however, cannot come at the cost of the underlying security. It is the topic of this research to explore the concept of **side channel attacks** that are enabled specifically when certain efficiency improvements are realized. We present to the reader **elliptic curves** in **Edwards form,** which carry several important and novel features that can be leveraged to achieve a high level of information security at a reduced cost of resources compared to traditional protocols such as RSA. We will show that there are multiple facets to the security of a given scheme, in that while a protocol or crypto-system may carry guarantees and proofs of security, flaws in implementations of these protocols can render them completely insecure. Far more must be taken into consideration when writing secure software.

1.b. Elliptic Curves and abelian groups

We present to the reader a concise introduction to the topic of abelian groups. An abelian group E can be represented as $\{E, \cdot\}$. E is defined as a set of elements along with a binary operation denoted $\cdot$, which maps (a, b) in E to another element in E. Consider the following abelian group:

$$E_{\mathbb{Z}} := \{-\infty \ldots 1,2,3 \ldots \infty \mid \cdot\}$$

Assume $\cdot$ represents normal addition. Taking element 1 and element 2 and performing the operation $\cdot$ maps to element 3. Any element in $E_{\mathbb{Z}}$ $\cdot$ any other element in $E_{\mathbb{Z}}$ gives an element in $E_{\mathbb{Z}}$.

We can define E to be a group containing the set of points on an elliptic curve. Consider the following equation:

$$x^2 + y^2 = 1 + dx^2y^2$$

This is an elliptic curve in Edwards form, labelled as E_{521} . We can solve for x and y to receive a coordinate pair which represents a point on this curve.

For this research, we utilize a set of specific security parameters:

$d = -376014$

$p := 2^{521} - 1$, a Mersenne prime defining the finite field $\mathbb{F}_p$, noting that (x, y) and their coefficients are exclusively members of $\mathbb{F}_p$. All curve operations are carried out modulo p .

$G := (x_0, y_0)$, referred to as the public generator point. Here, $x_0 = 4$, and y_0 is obtained by solving the curve equation above for x and y . G is a publicly known parameter to all participants in this scheme. Multiplying any secret scalar s by G yields sG. $\{s, sG\}$ is a key-pair including the private key s and the public key sG.

$\mathcal{O} := (0,1)$, which is the neutral point of addition. Any point $P + \mathcal{O} = P$, similar to how 1+0 = 1 for the integers in $\mathbb{Z}$. This element is useful as an initializing value for any variables used in our point operation algorithms.

How these parameters are derived is outside of the scope of this research, but a reference is left for the reader.

Since our elliptic curve is an abelian group of points, it is also necessary to define the binary operation. Here, we call it addition. Addition of any two points on our curve is carried out as the following:

$$(x_1, y_1) + (x_2, y_2) = \left(\frac{x_1 y_2 + y_1 x_2}{1 + d x_1 x_2 y_1 y_2}, \frac{y_1 y_2 - x_1 x_2}{1 - d x_1 x_2 y_1 y_2} \right)$$

(Note that all operations are modulo p)

With these parameters and a binary operation in hand, we now have an elliptic curve in which we can proceed to conduct an analysis on. We will show the differences in runtime between naive approaches to curve operation implementations, and we will test the security of the point multiplication algorithm as explored further on. A fully functional software implementation of this curve is presented in the java programming language.

We will present different approaches to curve operations and use this implementation to show the differences in runtime and resource consumption between them. It is these very differences which can render useless the security guarantees that this construction offers, and we will highlight these in greater detail in the coming sections.

2. Elliptic Curve point multiplication

Recall the point addition operation:

$$(x_1, y_1) + (x_2, y_2) = \left(\frac{x_1 y_2 + y_1 x_2}{1 + d x_1 x_2 y_1 y_2}, \frac{y_1 y_2 - x_1 x_2}{1 - d x_1 x_2 y_1 y_2} \right)$$

We can use repeated additions to serve as **multiplication**, and this operation is the focus of our research. The intrinsic value of an elliptic curve construction is that it is easy to compute a point addition any arbitrary number of times, but it is difficult to determine how many additions have been carried out simply by examining an arbitrary point. This is demonstrated as the following:

The number of points on our curve is $n := 4r$, where
$r = 2^{519} - 337554763258501705789107630418782636071904961214051226618635150085779108655765$

1. Select any integer s modulo n
2. compute sP, where P is any point included in our curve and multiplication is defined as repeated point additions:

$$sP = P_0 + P_1 + P_2 + \ldots + P_s$$

Given only sP, one would need to find the discrete logarithm of sP in order to recover s. This the discrete logarithm problem, and it is currently known to be intractable or infeasible to compute in a reasonable amount of time on classical computing machines. Thus, s can be viewed as a secret key. It is undesirable that s can be recovered from sP in a reasonable amount of time as this breaks the security of any scheme based on elliptic curves. Thus, the multiplication algorithm used to compute sP must be designed with great care and consideration.

Point multiplication represented as code:

```
multiply(s, P):

      let product = 𝒪;      #init to neutral point
      for i up to s:
            product = product + P
      return product

---------------------------------------
```

fig 1. Naive multiplication through repeated point addition

This algorithm has a time complexity of $O(s)$, since it requires s additions in order to compute sP. Considering that s is often an extremely large number, this approach leads to high runtimes that are undesirable in practice. We utilize our implementation of this approach to demonstrate the rapid increase in running time for larger values for s. For the runtime test carried out here, we use values 10, 100, 1000, 10000, and 100000 for s, and the generator point G as our input P.

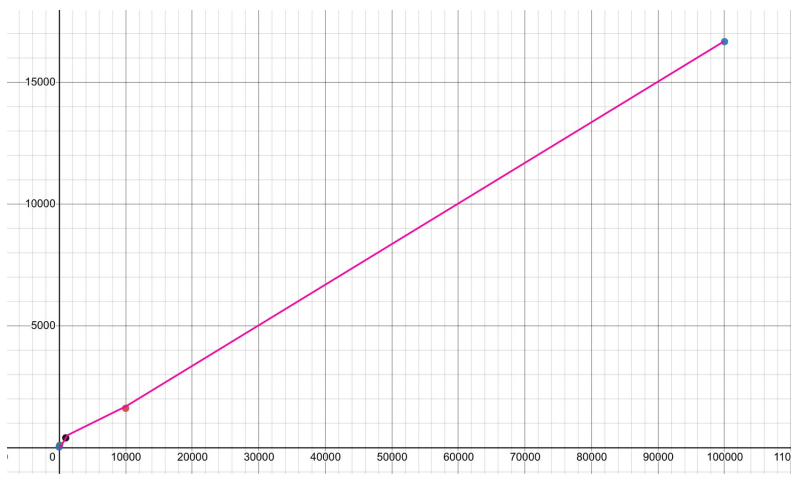


fig 2. Running time of `multiply` in nanoseconds. x-axis represents the value for s. y-axis is the running time in milliseconds. Here, 100,000 point additions is shown to require about 17 seconds to complete.

An improvement on this running time can be realized by implementing the **double and add** approach, analogous to the the exponentiation by squaring technique used for integers.

Algorithm 2:

```
doubleAndAdd(s, P):

      let product = 𝒪;      #init to neutral point
      let i = s.bitLength   #get number of bits in s

      while i >= 0 do:

            product = product.add(product)   #double
            if s[i--] == 1 do:               #test value of bit at
                                             #i and decrement i
                  product = product.add(P)   #add
      return product:
-------------------------------------------
```

fig 3. Iterative double and add point multiplication. For each iteration of the loop, the point is doubled. If the bit at $s_i = 1$, then the add subroutine is called. Double and add could appear as a recursive self-reduction as well.

The running time of this algorithm in the worst case is $O(log_2 s)$. We present a runtime analysis using the same values of s as above in the naive approach:

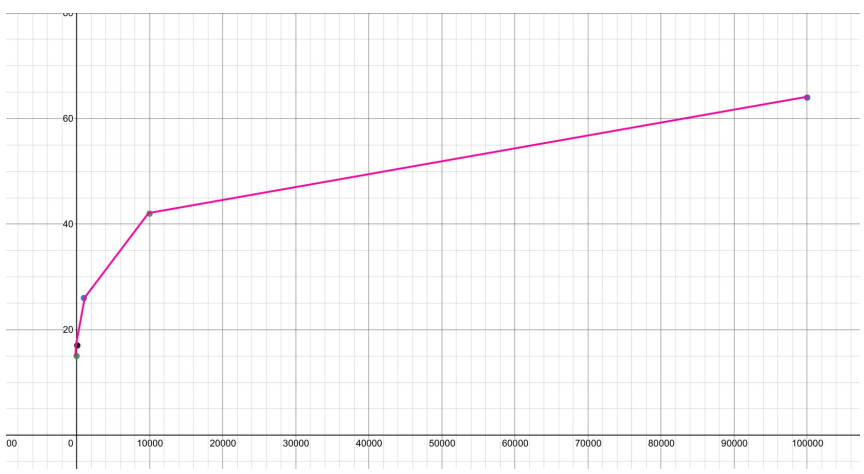


fig 4. A significant improvement in running time has been achieved. $log_2(100{,}000)$ point additions can be carried out in about 65 milliseconds. The logarithmic curve is clearly visible.

We reemphasize the importance of improvements in running time such as this. All software running behind cryptographic protocols benefit from this type of achievement. However, this approach to point multiplication has a fatal flaw that if ignored, will render the security assumptions of our elliptic-curve based scheme useless.

3. Side-channel attacks

A side channel attack is defined as any weakness in a piece of software that can be exploited by means other than the actual software itself. In this section, we present the different approaches to such attacks by examining the hardware on which the algorithm is executed.

The **multiply** algorithm exists in the same form for integers as it does for elliptic curves. One may notice that since the running time of this algorithm is directly proportional to the input size, a timing vulnerability may exist, in that the value of s can be inferred from the time it takes to calculate sP. While this may be true for integers, the very design of the Edward curve addition formula prevents such analysis. The importance of conducting operations modulo p becomes apparent here. Using our naive multiplication algorithm, we conduct the following:

$1 \times G =$

x coordinate: 4

y coordinate:
38323655458445572552474074524922764896486608619568788548234284097964210088578782437276
34210686791507168493732364650567836638782015285562571568855063494635788

$2 \times G =$

x coordinate:
14871919861657382182159267063136471680438842163025131320398814225662313774437906425528
87006224660371761500336911070673600217096760034811808787009321228744200

y coordinate:
61829357543620710123527195252596149372376730237707278435447928275650267135310284219278
46254119766235803901838311062589152803952092227098708714042880771649657

These results show the effective function of the "scattering" of the Edwards point addition formula. There is no apparent relation between an output produced from a value for $s = 1$ or $s = 2$, effectively mitigating an execution time side-channel attack. We proceed to examine another vector of attack in which power consumption of this algorithm can be analyzed, for which neither **multiply** nor **double and add** are protected against.

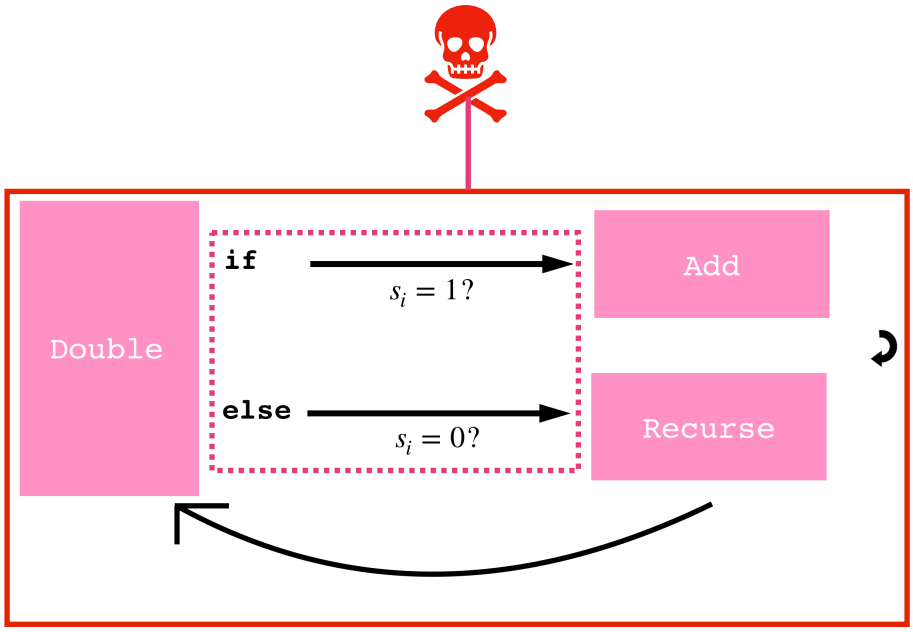
Recall Algorithm 2:

```
doubleAndAdd(s, P):

let product = 𝒪;          #init to neutral point
let i = s.bitLength       #get number of bits in s

while i >= 0 do:

      product = product.add(product)      #double
      if s[i--] == 1 do:                  #test value of bit at i and
                                          #   decrement i
            product = product.add(P)      #add
      else do: ?
return product:
-------------------------------------------
```

fig 5. Notice the **else** branch of the **if** statement.

Naive implementations of these algorithms only consider the high-level software details and disregard the low level circuitry in use. Here, it is important to understand that a well-equipped attacker can have access to the hardware on which this algorithm is executed.

Consider the following circuit:



Assume an attacker is able to measure the electric current on each branch of the **if** statement. They can use these measurements to see which branch is taken, and they can then see whether or not an **add** operation is performed. Knowing whether or not **add** is performed allows the attacker to recover the secret key s.

Consider a scalar value s of 153, which is represented in binary as 10011001. For each iteration **i** of the while loop, the attacker can record the following:

i	0	1	2	3	4	5	6	7
add?	yes	no	no	yes	yes	no	no	yes
s_i	1	0	0	1	1	0	0	1

add? is a test of voltage to see if an **add** operation was performed, while s_i is the attacker's guess of the bit value of s at iteration i of the loop.

Knowing which branch of the **if** statement that has been taken allows the attacker to fully recover the secret key s, if we assume that the attacker is also able to see if an **add** operation was carried out. This renders the double and add point multiplication algorithm insecure against an adversary with access to the low-level hardware of the algorithm.

As it turns out, power consumption and execution time analysis are not the only side-channel vulnerabilities that exists for elliptic curve point multiplication. It is also possible to conduct an analysis of which areas of memory are in use during execution of the algorithm. This research focuses only on the power consumption side-channel. A somewhat simple mitigation to the power consumption analysis attack exists in the form of the Montgomery Ladder:

```
MontgomeryLadder(s, P):

      let i = s.bitLength     #get number of bits in s
      let R0 = 𝒪;
      let R1 = P;

      while i >= 0 do:
            if s[i] do
                  R1 = R0.add(R1)
                  R0 = R0.add(R0)
            else do:
                  R0 = R0.add(R1)
                  R1 = R1.add(R1)
      return product:
-----------------------------------------
```

fig 6. The Montgomery ladder computes sP in a fixed number of operations, mitigating the power consumption side-channel. No matter the value of s_i , an **add** operation is always executed.

This algorithm effectively mitigates an analysis on electricity consumption of a given circuit, because no matter which branch of the **if** statement is taken, the same number of operations are executed, therefore consuming the same amount of electricity regardless of the value of the bits in the input. A detailed example is presented below:

Suppose s is set to 170, which in binary is represented as 10101010 . The attacker's analysis on doubleAndAdd would appear as:

i	0	1	2	3	4	5	6	7
add?	yes	no	yes	no	yes	no	yes	no
s_i	1	0	1	0	1	0	1	0

and in the Montgomery Ladder:

i	0	1	2	3	4	5	6	7
add?	yes	yes	yes	yes	yes	yes	yes	yes
s_i	1	1	1	1	1	1	1	1

The attacker would have 11111111 as the secret key, which $= 255 \neq 170$, therefore mitigating the power consumption side-channel.

We also present a run-time analysis of the Montgomery Ladder:

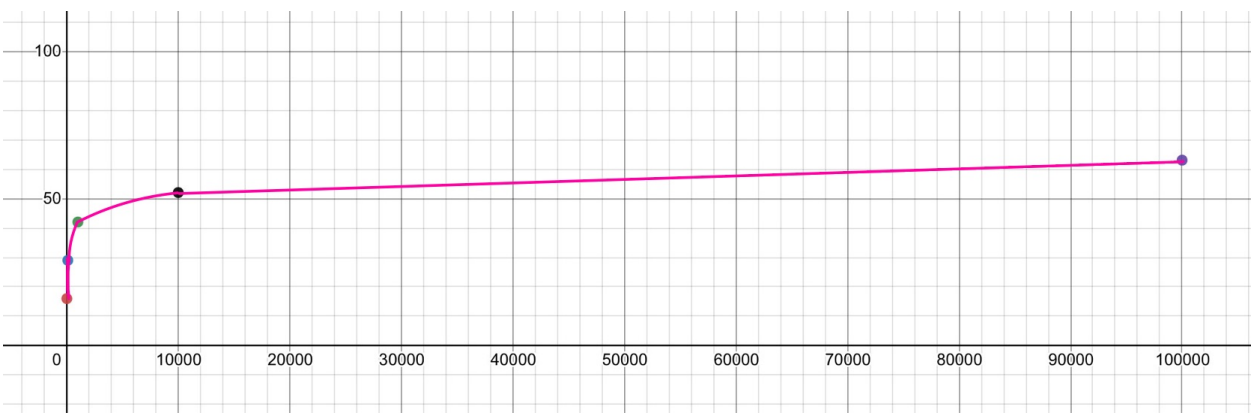


fig 7. The logarithmic runtime of the Montgomery ladder is clearly visible. Our algorithm can complete in about 52 milliseconds. The extra overhead is amortized for large values of s and has not experimentally affected our runtime.

4. Conclusion

There are additional details that are worth noting in this presentation. Although our algorithms are labelled as "point multiplication algorithms", it is important to understand that two points on an elliptic curve (at least in this construction) are never multiplied by each other. That is, if P is any valid point belonging to the curve E_{521}, we do not conduct multiplication as $P \times P$. We only ever multiply points by using repeated additions, with the number of additions dependent on the value of s. This is because the elliptic curve is an abelian group, and multiplication between two points is not defined.

Additionally, in our experimentation, we use the generator point G as the the input for our multiplication algorithms, although any valid P belonging to the curve can be used. The utility of using an arbitrary P becomes apparent in a key-exchange scheme. It is also important to note that our experimental input for s was chosen as successive powers of ten in order to highlight the differences in performance. A practical value for s is typically extremely large, for example:

$$s = 6864797660130609714981900799081393217269435300143305409394463459185543183397654701903506606654631398546774636260936570417277131794810169271973685172895668982$$

is an example of a valid private key for use in this crypto-system. This was calculated in the following steps:

1. Generate a random integer k of any length (SecureRandom from java was used here).
2. Calculate $s = k \% n$, recalling that $n := 4r$

Next, the finite field $\mathbb{F}_p$ that was presented along with other security parameters is distinct from the abelian group used to define the elliptic curve. $\mathbb{F}_p$ is instead used to define the set of values for which the coefficients of and values for (x, y) belong, and it also ensures that all operations carried out during point addition are done modulo p. This is important because elliptic curves are in practice represented on machines with finite memory. Utilizing a prime field such as $\mathbb{F}_p$ ensures that all curve operations produce results which can be stored in the memory of a computer without overflow.

We have shown in this analysis that improvements in efficiency and running time of an arbitrary algorithm can lead to undesirable side-effects that may not be immediately obvious. Careful planning must be taken when software is implemented, and far more than just the function of the code itself should be considered. The underlying hardware, specific abilities of an adversary, and many other factors must also be considered. Fortunately, in many applications there exists clever algorithmic approaches that can achieve the same gains in efficiency while mitigating any potential damaging side effects.

References:

1. Yarom, Y., & Benger, N. (2014). Recovering OpenSSL ECDSA Nonces Using the FLUSH+RELOAD Cache Side-channel Attack. IACR Cryptology EPrint Archive.

2. Aranha, Diego & Barreto, Paulo & Pereira, Geovandro & Ricardini, Jefferson. (2013). A note on high-security general-purpose elliptic curves. https://eprint.iacr.org.

3. Montgomery, Peter L. (1987). "Speeding the Pollard and elliptic curve methods of factorization". Math. Comp. 48 (177): 243–264. doi:10.2307/2007888. JSTOR 2007888. MR 0866113.

Repository for source code can be found at:

https://github.com/dustinvonsandwich/E521